ContentForge Project Documentation

Version: 0.1.0

Repository: `external_marketing_flow`

Generated on: 2026-05-23

1) Project Overview

ContentForge is an event-driven, human-in-the-loop AI content pipeline for social media marketing workflows.

It takes weekly research input, extracts and scores topics, creates a weekly plan, deep-researches selected topics, generates content per format (carousel, reel, single image/news post), supports iterative editing, and exports final assets.

Core goals:

- Keep research and planning structured
- Produce publish-ready content faster
- Preserve editorial control with explicit user checkpoints
- Track all artifacts by week and phase on disk

2) High-Level Architecture

Main runtime layers:

- Frontend: React + Vite UI (`frontend/`)
- Backend API: FastAPI (`api/`)
- Workflow engine: LangGraph state machine (`src/contentforge/core/graph.py`)
- Node implementations: modular generator/parser/scorer/export units (`src/contentforge/nodes/`)
- Persistence:

- LangGraph checkpointer (in-memory for thread state)
- File-based artifact memory (`data/weeks/...`)
- SQLite index support in core DB module

Primary execution model:

1. Frontend calls `/pipeline/start` with `week_id`
2. Graph runs until an interrupt node (human input required)
3. Frontend submits feedback/input using `/pipeline/{week_id}/feedback`
4. Graph resumes, pauses again at next interrupt, or finishes export

3) Repository Structure

Top-level directories:

- `api/`: FastAPI app, routes, request/response schemas, dependency wiring
- `src/contentforge/core/`: graph, state model, config, logger, memory, DB, prompt loading
- `src/contentforge/nodes/`: all pipeline node logic grouped by phase
- `prompts/`: prompt templates for research, deep research, content, editing
- `frontend/`: operator dashboard and interactive workflow UI
- `carousel_renderer/`: renderer service dependency for carousel image previewing
- `data/`: runtime artifacts, logs, brand context
- `tests/`: unit/integration tests for nodes and flows
- `scripts/`: local operational scripts (run node, fire event, tests, polling, etc.)

4) Backend API Design

Base server:

- App entrypoint: `api/main.py`
- CORS: wide-open for local dev (`allow_origins=["*"]`)
- Health check: `GET /health`

4.1 Pipeline Routes (`api/routes/pipeline.py`)

- `POST /pipeline/start`

- Starts a fresh or reset pipeline thread for a `week_id`
- Initializes `ContentForgeState`
- Invokes graph until first interrupt
- Returns state + required human action + prompt content

- `GET /pipeline/{week_id}/status`

- Returns current graph/thread snapshot status
- Indicates current human action type, pending topic, and prompt content

- `POST /pipeline/{week_id}/feedback`

- Resumes interrupted graph with structured actions:
- `supply_raw_research`
- `select_topics`
- `supply_deep_research`
- `approve`, `approve_content`, `edit`, `approve_plan`
- Applies state update and resumes graph with timeout protection

Human-interrupt mapping currently used:

- `research_parser` -> `paste_research`
- `deep_prompt` (after planner interrupt) -> `select_topics`

- `deep_parse` -> `paste_deep_research`

4.2 Memory Routes (`api/routes/memory.py`)

- `GET /memory/artifact/{week_id}/{phase}/{filename}?topic_id=...`

- Reads generated artifacts from file memory
- Returns markdown/json artifact content + metadata

- `GET /memory/brand/context`

- Returns brand context currently loaded from `data/brand`

4.3 Events Route (`api/routes/events.py`)

- `WS /events/ws`

- Tails `pipeline_events.log`
- Streams normalized event payloads to frontend for live telemetry

4.4 Carousel Route (`api/routes/carousel.py`)

- `POST /carousel/render/{week_id}/{topic_id}`

- Pulls rendered JSX code from graph state
- Calls external renderer service (`CAROUSEL_RENDERER_URL`, default `http://localhost:4000`)
- Returns base64 data URLs for slide preview images

5) State Model and Contracts

Canonical state definition:

- `src/contentforge/core/state.py`
- Typed via Pydantic models and enums

Important enums:

- `PipelinePhase`: idle, research, scoring, planning, deep_research, content_creation, review, export
- `ContentFormat`: carousel, single_image, reel, news_post
- `ContentStatus`: pending, draft, editing, approved, exported
- `HumanActionType`: paste_research, select_topics, paste_deep_research, review_content, chat_edit

Important nested models:

- `Topic`, `PlanItem`, `DeepResearchItem`
- `Theme`, `Caption`, `CarouselSlide`
- `TopicContent` (all output per topic)
- `ContentForgeState` (single source of truth for graph)

6) Workflow Graph (LangGraph)

Graph builder:

- `src/contentforge/core/graph.py`

Entrypoint and phases:

1. `brand_loader`
2. `prompt_gen`
3. `research_parser` (interrupt before for manual research paste)
4. `scorer`
5. `planner` (interrupt after for topic selection)
6. `deep_prompt`
7. `deep_parse` (interrupt before for deep research paste)
8. `content_router` -> format branches
9. `theme_designer` -> `caption_writer`
10. `edit_router` / `chat_edit` loop
11. `export_agg` -> `packaging`

Conditional routers:

- `format_router`: chooses carousel/reel/single-image path per topic
- `carousel_router`: loops slide writing and code generation until done
- `deep_research_router`: loops through topic queue
- `edit_action_router`: export vs chat-edit
- `post_packaging_router`: continue next selected topic or end

Interrupt behavior:

- `interrupt_before=["research_parser","deep_parse"]`
- `interrupt_after=["planner"]`

7) Node Layer Responsibilities

7.1 Research Nodes

- `brand_context_loader.py`: loads brand + platform constraints
- `research_prompt_generator.py`: builds prompt packet for top-of-funnel research collection
- `research_parser.py`: turns raw research text into normalized topic bank

7.2 Scoring & Planning Nodes

- `topic_scorer.py`: scores extracted topics by relevance/value
- `calendar_planner.py`: schedules selected topics into weekly plan entries

7.3 Deep Research Nodes

- `deep_research_prompt_generator.py`: builds per-topic deep research instructions

- `deep_research_parser.py`: normalizes deep research output into structured content specs

7.4 Content Nodes

- `content_router.py`: initializes/dispatches topic content generation by format
- `theme_designer.py`: visual language and typography payload per topic
- `caption_writer.py`: platform-specific caption variants
- `image_prompt_engineer.py`: image prompts for image-led formats
- Carousel subflow:

- `carousel_creator.py`
- `slide_content_writer.py`
- `react_code_generator.py`

- Reel subflow:

- `script_writer.py`

7.5 Editing + Export Nodes

- `edit_router.py`: decides whether human changes are pending
- `chat_edit_agent.py`: targeted re-generation loop from editor feedback
- `content_aggregator.py`: consolidates content for final output
- `file_packager.py`: writes/organizes export artifacts

8) Frontend Application

Main files:

- `frontend/src/App.tsx`
- `frontend/src/pages/Dashboard.tsx`
- `frontend/src/pages/CalendarView.tsx`
- `frontend/src/components/HumanInputPanel.tsx`
- `frontend/src/components/HumanInLoopChat.tsx`
- `frontend/src/components/ContentBoilerplate.tsx`
- `frontend/src/services/api.ts`

Key frontend behavior:

- Starts and tracks weekly pipeline runs
- Shows interrupt prompts and required human actions
- Allows topic selection and deep-research submissions
- Renders content output previews
- Exports slides as JPGs/ZIP from carousel/single view
- Consumes websocket stream for live pipeline event updates

9) Data and Artifacts

Artifact patterns:

- Week-scoped outputs in `data/weeks/<week_id>/<phase>/...`
- Topic-scoped files for deep research/content phases
- Brand context in `data/brand/*`
- Event logs used for live UI stream

Artifact use-cases:

- Rehydrating prompts on refresh via memory endpoints
- Debugging parser/scorer decisions
- Preserving auditable generation history

10) Configuration and Environment

Python config:

- `pyproject.toml` declares Python 3.12+, LangGraph/LangChain/FastAPI/OpenAI stack

Environment variables (typical):

- `LLM_BASE_URL`
- `LLM_API_KEY`
- `LLM_DEFAULT_MODEL`
- Optional tracing:

- `LANGCHAIN_TRACING_V2`
- `LANGCHAIN_API_KEY`
- `LANGCHAIN_PROJECT`

- Carousel renderer:

- `CAROUSEL_RENDERER_URL` (default `http://localhost:4000`)

Frontend environment overrides:

- `VITE_API_HOST`, `VITE_API_PORT`, `VITE_API_PROTOCOL`
- `VITE_API_BASE_URL`, `VITE_WS_BASE_URL`

11) Local Development Workflow

Backend:

1. Activate environment
2. Install deps (`uv sync`)
3. Run API (`uv run uvicorn api.main:app --reload --port 8000`)

Frontend:

1. `cd frontend`
2. `npm install`
3. `npm run dev`

Renderer (if using carousel previews):

1. Start renderer service on configured URL/port

Tests:

- `uv run pytest tests/ -v`

12) Quality and Test Coverage

Observed test organization includes:

- Node-level tests for research parser/scorer/content router/deep parser
- Topic stability and parser scripts in `scripts/` for flow validation

Recommended test priorities:

- API contract tests for feedback actions and interrupt transitions
- End-to-end tests validating full multi-topic loop
- Export tests for carousel/single-image slide capture and download behavior

13) Operational Notes

Current strengths:

- Explicit human checkpoints reduce bad autonomous drift
- Modular node boundaries make targeted fixes easier
- Strong week/topic scoping for artifacts and reproducibility

Current operational risks to monitor:

- Long-running LLM calls or renderer latency
- Missing deep research input for selected topics
- Frontend/browser download restrictions for bulk exports

Hardening recommendations:

- Add persistent checkpointer (Redis/DB-backed) for multi-process resilience
- Add idempotency keys for start/resume operations
- Add structured error codes in API responses
- Add integration dashboard for node latency and fail counts

14) Troubleshooting Quick Guide

- `Pipeline start failed: Network Error`

- Verify API process is running and reachable on expected host/port
- Check CORS/protocol mismatch between frontend and backend

• Research/deep-research appears stuck

- Query `GET /pipeline/{week_id}/status` and inspect `human_action_type`
- Submit corresponding `/feedback` action payload

• Carousel preview fails

- Verify `CAROUSEL_RENDERER_URL` service is up
- Confirm `rendered_code` exists for the selected topic

• Export button does not trigger file

- Browser may block delayed auto-downloads
- Use manual fallback download link in UI

15) Roadmap Suggestions

- Add multi-workspace tenancy and access control
- Add prompt versioning + rollback by phase
- Add declarative policy checks for unsafe/low-quality outputs
- Add content analytics feedback loop into topic scorer

This document reflects the current codebase state in `external_marketing_flow` as of 2026-05-23.